

API Security Risk Analysis Report

Fatima Ilyas

22/06/2026

Table of Contents

Non-Disclosure Agreement 3

Legal Notice 3

Abstract..... 3

Introduction..... 4

Methodology & Documentation..... 4

 I. Unauthorized Access to User Data 4

 II. Excessive Data Exposure 5

 III. Weak Authentication Mechanisms 5

 IV. Poor Error Handling..... 6

 V. Lack of Rate Limiting 6

Header Analysis 7

Conclusion 7

References..... 8

Non-Disclosure Agreement

All data analyzed, and findings presented in this report, are derived from publicly available, or intentionally provided test APIs, and are used strictly for educational, and security awareness purposes. No real user data, confidential information, or private systems, were accessed, tested, or exposed, during this assessment.

Legal Notice

This report is based on the analysis of a publicly available test API (ReqRes), using standard API testing techniques. The findings presented are intended for educational purposes only, and reflect simulated security risks, rather than vulnerabilities, in a production system.

Abstract

This report presents a read-only API security risk analysis conducted on the ReqRes API, a publicly available test platform, designed for learning, and practicing API interactions. The purpose of this assessment is to identify common security risks, associated with modern APIs, including authentication weaknesses, excessive data exposure, and improper error handling.

The analysis was performed using API testing tools such as Postman, focusing on inspecting request methods, headers, and server responses, without attempting to exploit, or disrupt the system. Several security concerns were identified, including unauthenticated access to user data, lack of rate limiting, and weak token-based authentication mechanisms.

This report highlights the potential impact of these risks, and provides practical remediation strategies, aligned with industry best practices. The findings emphasize the importance of securing APIs in modern applications.

Introduction

In modern web applications, APIs play a crucial role in enabling communications between different systems, including mobile applications, web platforms, and third party services. Application Programming Interfaces, or APIs, are constructs of complex code, made available in programming languages, to allow developers to create intricate functionality more easily. They abstract the more complex code away from application programmers, and provide some easier syntax, which contains that complex code to process the same function, to use in its place [1]. Because APIs handle sensitive data, and core application functionality, they have become a significant target for attackers.

This report focuses on conducting an API security risk analysis of the ReqRes API, a publicly accessible testing platform. The objective of this assessment is to evaluate the security posture of the API by analyzing its endpoints, request structures, authentication mechanisms, and response data. By performing this examination, the report aims to record the demonstrations of fundamental API security concepts, and highlight the importance of implementing strong authentication, proper data handling, and secure API design.

Methodology & Documentation

This API risk analysis was conducted using a structured, and systematic approach. First the API documentation provided by ReqRes was reviewed [1], followed by testing endpoints provided by ReqRes' playground using Postman, sending GET, and POST requests using the provided x-api-key by ReqRes, to observe behavior, inspecting request, and response data, and evaluating authentication, and authorization mechanisms, and discovering potential security risks based on observed responses. The analysis focused on identifying security weaknesses, rather than exploiting vulnerabilities.

I. Unauthorized Access to User Data

Screenshots "get1", and "get2", show GET requests performed, which return sensitive user information to unauthorized users, without requiring authentication. The endpoints used were "/api/users/2", which return information about the user with the id 2, and "/api/users?page=2", which returns the second page of user information from the user's database. These endpoints can be tested, and accessed directly, without requiring any tokens, or logins. This unauthorized access to user data increases the risk of data scraping (the process of pulling information out of a website, and into a spreadsheet, for analysis, processing, or presentation [2]), and can lead to privacy violations, and potential legal issues. The security vulnerability associated with this risk, is the Broken Object Property Level Authorization, or BOPLA, where an application exposes, or allows modification of specific object properties, that users should not access [3].

Risk Level: High

Authentication processes, such as the implementation of biometric authentication, which uses fingerprints, faces, palm prints, voices, or other biological, and behavioral features, multi-factor authentication, object property-level authentication for BOPLA attacks [4], JWT (JSON Web Token), which is a token format, used to securely transmit data, and handle stateless information, and OAuth, which is an authorization framework, used to grant an application limited access to user resources, without exposing credentials [5], and restricting access to user data, based on authorization levels, reduces the likeliness of this vulnerability from occurring.

II. Excessive Data Exposure

Screenshots “get3”, and “reqres docs”, show the GET requests performed to gain access to application information which should not be accessed by users, along with screenshots “get1”, and “get2”, which provide excessive user information, that does not need to be accessed by any user. The API responses include multiple user attributes, such as email, full name, and avatar, regardless of necessity. This unnecessary exposure of user, and system information, increases the risk of phishing, and social engineering attacks, along with the risk of hackers, exploiting sensitive data by sniffing traffic (capturing, and viewing network traffic as it is transmitted, intercepting, and analyzing clear text network packets, which can provide unauthorized access to systems [6]).

Risk Level: Medium

A range of measures can be implemented, such as the ensuring of authorization being implemented correctly, limiting API responses to only required data fields, and applying data minimization principles, including purpose limitation (data should only be collected for specified, explicit, and legitimate purposes, and cannot be further processed in a manner that is incompatible with those purposes), storage limitation (personal data should be kept in a form which permits identification of data subjects for no longer than is necessary for the purposes for which the personal data are processed), and accountability (the organization (the data controller) is responsible for, and must be able to demonstrate, compliance with all of these principles) [7].

III. Weak Authentication Mechanisms

Screenshots “post1”, and “post2”, show the POST requests performed to login, into an account, and register for an account. Both the login, and register endpoints return a token (device-specific unique code that features user-specific information that follow a standard authorization and authentication process while forming a domain [8]), immediately after successful login and registration, without demonstrating proper validation, expiration, or security controls, like multi-step verification. This vulnerability can cause tokens to be reused indefinitely, without expiration, causes lack of session management, which increases security risks, and poses a risk for unauthorized access if tokens are exposed.

Risk Level: High

It is recommended to implement secure token generation using JWT, adding token expiration, and refresh mechanisms to prevent unauthorized access, avoiding the issuing of authentication tokens immediately upon registration without proper authentication, and enforcing said proper authentication validation, including processes like two-factor, or multi-factor authentication.

IV. Poor Error Handling

When performing a POST request with a login endpoint, the API provides detailed error messages, such as “Missing password”, as shown in screenshot “posterror”, which may reveal information about expected inputs. This can lead to attackers understanding API behavior, and increased risk of targeted attacks. However, a response containing a complicated, and non-user-friendly error results in users getting frustrated, and abandoning the platform [9].

Risk Level: Medium

Performing best practices to handle API errors on both the server, and client side, are providing errors in a clear, and consistent structure, using descriptive error messages, avoiding the leaking of sensitive data, documenting common errors, using HTTP status codes properly, providing enough detail in error messages, using a standardized error response format, and implementing logging and monitoring, for server side error handling. Whereas on the client side, validating user input, providing user-friendly messages, and handling multiple edge cases, solves the error handling problem [10].

V. Lack of Rate Limiting

This API does not enforce request limits, allowing for unlimited, repeated requests. Rate limiting, also known as request throttling, controls the number of requests a client can make within a specified time window. This process protects backend systems from overload, while ensuring fair access for all consumers. Rate limiting serves several critical functions, including the prevention of abuse by stopping malicious actors from overwhelming the API, ensuring reasonable access for all users, managing infrastructure expenses by limiting unnecessary traffic, and keeping response times fast during high-traffic periods. Poor rate limiting can result in potential brute force attacks, API abuse, DDoS attacks, credential stuffing (automated login attempts using leaked credentials), web scraping (unauthorized data harvesting that consumes resources), and potential service disruption [11].

Risk Level: Medium

It is recommended to perform the following best practices, which include implementing rate limiting, for example, requests per minute, using throttling, and monitoring mechanisms, documenting time windows, and how to interpret headers, and tracking metrics like rate limit hit frequency, request volume distribution, patterns in violation, and impact on performance [11].

Header Analysis

This report examined the headers of each endpoint, and identified any anomalies found that could jeopardize the safety, and security of the API.

I. GET /api/users?page=2

This endpoint displayed all the users, and their information on the second page of the user database. The anomalies found were:

- **Access-Control-Allow-Origin: ***: any website can call this API, leads to potential data exposure
- **Ratelimit-Limit: 20 & X-Ratelimit-Limit: 250**: two different conflicting rate limit headers, leads to inconsistent security enforcement
- **Cache-Control: public**: sensitive data could be cached unintentionally to the public.

II. POST /api/login

This endpoint logs into a user account with a username and password. The anomalies found were:

- **Authorization** header absent, increased risk of unauthorized access
- **Security metadata** absent, only returns a token, token system too simple
- **Expiration, session binding, validation proof** absent
- **Rate Limiting** headers absent in the main response, only global headers present

III. GET /api/users/2

This endpoint displays a valid user, and their information, with the identification number 2. The anomalies found were:

- **Access-Control-Allow-Origin: ***
- **X-Reqres-Limit-Warning: approaching**: internal information leaked in headers, attackers learn system limits, and service state
- **X-Reqres-Trial-Days-Remaining & X-Reqres-Trial-Expires**: used for the tracking of the active premium testing trial

Conclusion

To conclude, this API risk analysis identified several significant security weaknesses within the ReqRes API environment, including unauthorized access to user data, excessive data exposure, weak authentication mechanisms, poor error handling, and the absence of rate limiting, along with the examination of headers,

and anomalies found. While the API is designed for testing, and educational purposes, these findings highlight real-world security risks, that could have serious implications if present in a real system. The analysis demonstrated how insufficient authentication, and authorization controls can lead to high-risk vulnerabilities, such as unauthorized data access, and token misuse. Additionally, issues such as excessive data exposure, and improper error handling further increase the attack surface, making it easier for adversaries to gather information, and exploit system behavior. The lack of rate limiting also exposes the API to abuse, including brute force, and DDoS attacks.

To mitigate these risks, it is essential to implement strong security measures, including robust authentication, and authorization mechanisms, data minimization practices, secure token management, standardized error handling, and effective rate limiting. Applying these best practices not only enhances API security, but also ensures the protection of user data, and system integrity.

Overall, this assessment reinforces the importance of secure API design, and proactive risk management. Even in controlled, or simulated environments, identifying, and understanding these vulnerabilities, is critical in preparing for real-world cybersecurity challenges.

References

1. MDN Web Docs. (2024). Introduction to web APIs - Learn web development | MDN. [online] Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Client-side_APIs/Introduction.
2. Okta (2023). What Is Data Scraping? Definition & Usage | Okta. [online] www.okta.com. Available at: <https://www.okta.com/identity-101/data-scraping/>.
3. Tyk API Management. (2024). API security risks and mitigation: Essential strategies to safeguard your APIs. [online] Available at: <https://tyk.io/learning-center/api-security-risks-and-mitigation-essential-strategies-to-safeguard-your-apis/>.
4. Kumar, R. (2026). Broken Object Property Level Authorization BOPLA Explained. [online] Levo.ai. Available at: <https://www.levo.ai/resources/blogs/broken-object-property-level-authorization>
5. supertokens.com. (n.d.). OAuth vs JWT (JSON Web Tokens): An In-Depth Comparison. [online] Available at: <https://supertokens.com/blog/oauth-vs-jwt>.
6. www.sciencedirect.com. (n.d.). Sniffing Traffic - an overview | ScienceDirect Topics. [online] Available at: <https://www.sciencedirect.com/topics/computer-science/sniffing-traffic>.
7. Kiteworks (2025). What is Data Minimization and Why is it Important? [online] Kiteworks | Your Private Content Network. Available at: <https://www.kiteworks.com/risk-compliance-glossary/data-minimization/>.

8. www.wallarm.com. (n.d.). What is an API token? 🗝️ Quick Guide. [online] Available at: <https://www.wallarm.com/what/what-is-an-api-token-quick-guide>.
9. Zuplo. (2025). API Error Handling That Won't Make Users Rage-Quit | Zuplo Learning Center. [online] Available at: <https://zuplo.com/learning-center/optimizing-api-error-handling-response-codes>.
10. Bello, G. (2024). Best practices for API error handling. [online] Postman Blog. Available at: <https://blog.postman.com/best-practices-for-api-error-handling/#what-are-some-best-practices-for-handling-api-errors-on-the-client-side>
11. Team, T.P. (2026). What is API Rate Limiting? Understanding Request Throttling and Best Practices. [online] Postman Blog. Available at: <https://blog.postman.com/what-is-api-rate-limiting/#section-7>
12. Dennis, C. (2026). APIs for Beginners - How to use an API (Full Course/Tutorial). [online] Youtu.be. Available at: https://youtu.be/WXsD0ZgxjRw?si=Tf4SkV-g_xzbyr1T